SYSTEM DESIGN DOCUMENT

Phishing URL Detection Module

High-Level Architecture, Module Boundaries, Data Pipeline and Flowchart

Student Name: JEEVITHA L SHETTY
SRN : R23EQ045

1. Introduction

This System Design Document translates the phishing URL detection module's requirements into a high-level architecture. It defines module boundaries, data pipelines, component interactions, interfaces, quality controls and evaluation. It is grounded in the supplied v2 roadmap: a URL-only prototype using character n-gram TF-IDF, a small NumPy character-CNN and an ensemble comparison; registered-domain-disjoint evaluation; recall-oriented thresholding; URL-padding robustness experiments; and lightweight explanations.

The architecture separates offline model development and evaluation from online prediction. This reduces leakage risk, supports reproducibility and allows the model to be replaced without changing the external interface. DNS, TLS, webpage content, Transformers and production deployment are not claimed as already implemented.

1.1 Purpose

Define a maintainable design for accepting a URL, applying a consistent feature pipeline, producing a phishing-risk prediction, applying a validated threshold and returning a result with appropriate metadata. Include an offline workflow for training, calibration, robustness tests and reporting.

1.2 Scope

- Current scope: URL validation and normalization; lexical/character features; TF-IDF baseline; optional CPU character-CNN and ensemble; thresholding; prediction response; lightweight explanation; domain-aware data splitting; evaluation and reporting.
- Future extensions: DNS age/records, TLS certificates, safe webpage features, PyTorch sequence models or a URL-pretrained Transformer, real temporal monitoring, and analyst evaluation of explanations.
- Not claimed: production readiness, true temporal/zero-day generalization without timestamped data, multimodal gains without implemented signals, or protection against adaptive attackers without adaptive testing.

2. Design goals and principles

- Traceability: map each functional requirement to components and acceptance checks.
- Separation of concerns: training, evaluation, prediction, explanation and interface/reporting are separate.
- Reproducibility: record data manifests, domain split manifests, model versions, configuration, seeds and dependency versions.
- Leakage prevention: group by registered domain before split; never use final test data to select thresholds.
- Recall-aware operation: select thresholds on validation/calibration data and report false-positive rate with recall.
- Safe handling: treat submitted URLs as untrusted text; do not automatically open them in the prediction path.
- Extensibility: future domain/page signals sit behind a feature-provider interface.
- Honest evidence: scope claims to datasets, splits and attacks actually evaluated.

3. High-level architecture

The system has two planes: (A) an offline model-development plane and (B) an online prediction plane. Offline processing audits labels and sources, creates registered-domain-disjoint splits, trains models, calibrates thresholds, runs robustness/generalization tests and produces a versioned model package and report. Online prediction validates an incoming URL, runs the matching feature pipeline and model, applies the selected threshold and returns a verdict with optional explanation.

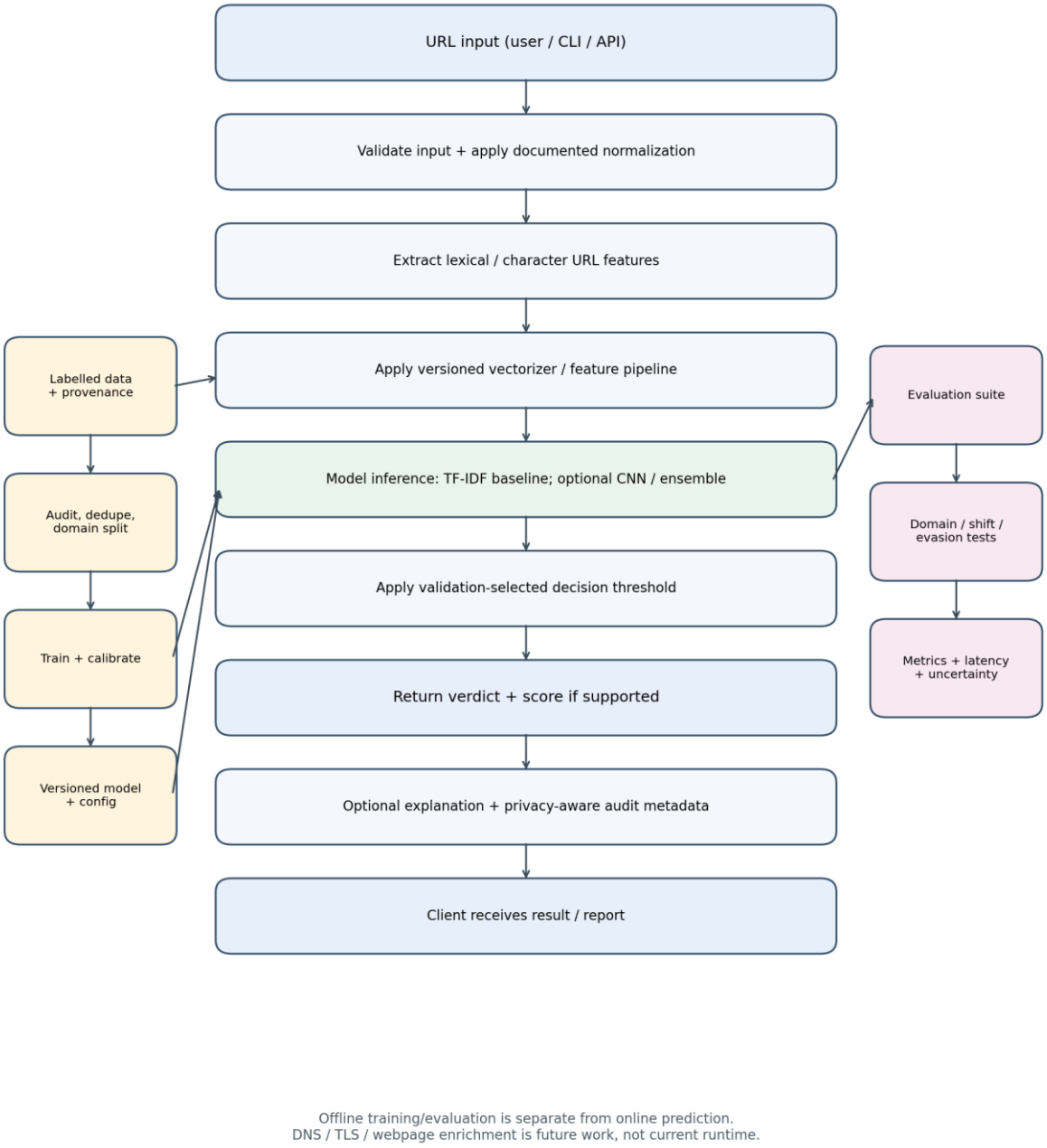


Figure 1. High-level flowchart showing offline training/evaluation and online prediction.

3.1 Architecture layers

Layer	Components	Responsibilities
Input / interface	CLI, API or UI adapter	Accept URL text, enforce input limits and return a consistent response.
Validation / preprocessing	URL validator, normalization policy, parser	Validate syntax and apply documented normalization.
Feature extraction	Lexical feature extractor, character n-gram vectorizer	Generate deterministic features identical to training.
Inference	Model adapter, model artifact, version/config loader	Load matching vectorizer/model and produce score or class.
Decision policy	Threshold and recall-control configuration	Apply threshold selected on validation/calibration data.

Explanation / audit	Feature indicators, linear explanation, CNN saliency, privacy-aware logger	Provide supported explanation and minimal metadata.
Offline data pipeline	Data loader, audit, deduplication, domain grouper, split builder	Prepare traceable data and prevent domain leakage.
Training / evaluation	Trainer, calibration, robustness test runner	Train candidates and compare on identical held-out splits.
Reporting / registry	Metrics exporter, manifest, model registry	Store model versions, evaluation evidence and reports.

4. Module boundaries and responsibilities

Module	Inputs	Outputs / responsibility
M1 URL input adapter	URL string / request	Accept URL text; return validated request or error. Does not fetch destination.
M2 Validator / normalizer	Raw URL text	Normalized representation; rules are versioned and shared with training.
M3 Dataset audit / split builder	Labelled URLs, source/domain metadata	Audited data and split manifest; domain groups cannot cross splits.
M4 Feature pipeline	Normalized URL	Feature vector or sparse matrix; shared between training and inference.
M5 Model inference adapter	Features + model artifact	Score/class; supports TF-IDF classifier and optional CNN/ensemble.
M6 Threshold policy	Raw score + threshold config	Final Phishing/Legitimate verdict and operating-point metadata.
M7 Explanation service	Features, model, prediction	Feature indicators/explanation; does not change the predicted label.
M8 Evaluation service	Model + held-out splits + transformations	Metrics, confidence intervals and latency results.
M9 Registry/report service	Model, config, metrics, dependencies	Versioned package and report; prevents model/vectorizer/threshold mismatch.

5. Data pipeline

5.1 Offline training and evaluation

- Acquire labelled phishing and legitimate URLs from permitted sources; record source, retrieval date if available, label definition, licence/access constraints and data version.
- Audit missing/malformed URLs, duplicates, class balance, source-specific shortcuts and related registered-domain overlap.
- Normalize and parse URLs with deterministic, versioned rules.
- Derive registered-domain identifiers using a maintained Public Suffix List implementation or equivalent.
- Create train, validation/calibration and test splits grouped by registered domain; save split manifests.
- Fit feature transformations on training data only and apply them unchanged to validation and test data.
- Train the TF-IDF baseline, then optional character-CNN and ensemble candidates using comparable splits.
- Select the threshold/recall-control policy on validation/calibration data; freeze it before final testing.
- Evaluate clean performance, domain-level metrics and documented URL-padding or other robustness tests.
- Export metrics, uncertainty estimates where available, latency, environment and limitations; version model, features and threshold together.

5.2 Online prediction

- Receive URL text through UI, CLI or API.
- Validate type and length; reject invalid input without attempting network access.
- Apply the same normalization and feature pipeline version used for training.
- Run the compatible model and calculate a score/class; do not call a score a calibrated probability unless calibration was assessed.
- Apply the configured threshold to produce a final verdict.
- Optionally produce a model-appropriate local explanation.
- Return verdict, optional score, model version, policy identifier and concise explanation metadata.
- Log only necessary operational metadata; avoid retaining full URLs/query strings unless required and governed by a retention policy.

6. Component interactions and interfaces

6.1 Suggested request/response contract

This is a logical interface for the design, not a claim that a web API is already implemented.

Field	Meaning
Request.url	Required string containing the submitted URL.
Response.label	Required categorical result: Phishing or Legitimate.
Response.score	Optional model score where supported.
Response.score_semantics	Clarifies decision value, uncalibrated score or calibrated probability.
Response.model_version	Identifier for model and feature pipeline package.
Response.policy_version	Identifier for threshold/recall-control policy.
Response.explanation	Optional feature indicators for supported models.
Response.warning	Optional statement that prediction is a risk estimate, not a guarantee.

6.2 Model package integrity

The model artifact must be paired with its vectorizer/feature schema, normalization version, threshold configuration, class mapping, dependency versions and evaluation manifest. If any component is missing or incompatible, the service should fail safely rather than return a potentially incorrect prediction.

7. Functional requirement traceability

Task 1 functional requirement	Architectural component(s)	Verification / acceptance evidence
FR-1 Accept URL input	M1 Input adapter	Valid input accepted; missing/wrong-type input returns controlled error.
FR-2 Validate and normalize	M2 Validator/normalizer	Unit tests for malformed, Unicode, long and edge-case URLs.
FR-3 Extract URL features	M4 Feature pipeline	Feature shape and deterministic-output tests.
FR-4 Optional domain/DNS features	Future feature-provider interface	Not enabled in current URL-only prototype; if added, test safety, latency and missing values.
FR-5 Optional webpage/HTML metadata	Future isolated enrichment service	Not implemented currently; require safe fetching and legal/privacy review.
FR-6 Preprocess feature types	M4 Feature pipeline	Same fitted transformations used in training and inference.
FR-7 Feature selection/vectorization	M4 Feature pipeline	Versioned vectorizer/schema and regression tests.
FR-8 Classify URL	M5 Inference + M6 threshold	Held-out classification report and end-to-end test.
FR-9 Return score/risk	M5 + response serializer	Score semantics documented; calibration tested if probability is claimed.
FR-10 User-friendly result	M1 interface/response serializer	Usability and error-state checks.
FR-11 Explanation / feature indicators	M7 Explanation service	Supported explanation generated; latency/limits documented.
FR-12 Logging/auditing	M7 audit logger + report service	Logs contain needed version metadata without unnecessary URL retention.
FR-13 Controlled retraining	Offline pipeline + registry	Retraining creates candidate version; evaluation required before promotion.

8. Non-functional design considerations

Quality attribute	Design decision	Measurement
Performance	Keep current online path URL-only; avoid synchronous network calls.	Median and p95 prediction latency on target CPU.
Reliability	Deterministic preprocessing, error handling, versioned artifacts.	Unit/integration tests and repeat-run consistency.
Generalization	Registered-domain-disjoint splits; independent data where feasible.	Domain-level recall, precision, F1, PR-AUC and FPR.
Robustness	Separate offline transformation suite; document threat model.	Clean-vs-transformed metric changes.
Reproducibility	Save split manifests, seeds, provenance, configs and package versions.	Documented rerun within expected variation.
Security	Treat URL as untrusted text; do not automatically navigate to it.	Input tests and security review of any future fetcher.
Privacy	Minimize raw URL logging and define retention.	Log/retention review.
Maintainability	Separate input, features, model, threshold, explanation and evaluation.	Module tests and documented interfaces.
Scalability	Support batch inference and replaceable model adapters.	Throughput and memory tests.
Explainability	Expose explanations only where supported.	Latency and, if claimed, faithfulness/usefulness evaluation.

Adaptability	Promote new models only after evaluation; keep rollback version.	Versioned release and rollback test.

9. Evaluation and monitoring

The evaluation service reports accuracy, precision, recall, F1, confusion matrix, PR-AUC or another justified score-based metric, false-positive rate and latency. For a recall-first goal, report recall alongside FPR at the selected operating point. Distinguish URL-level and registered-domain-level summaries because a few large platforms can dominate URL counts. Use repeated seeds and paired uncertainty estimates where feasible; cluster bootstrap by registered domain is appropriate when multiple URLs belong to one domain.

The supplied v2 roadmap reports three seeds and registered-domain-disjoint splits. Further evidence should include more split seeds, a 1,000-resample cluster bootstrap, paired confidence intervals, multiple-comparison handling and a second public URL dataset. The supplied roadmap says trustworthy timestamps are unavailable, so this design does not claim temporal generalization or zero-day performance.

9.1 Current prototype status

Capability	Status in supplied roadmap	Design implication
TF-IDF / character n-gram	Implemented main baseline	Primary lightweight reference.
NumPy character-CNN	Implemented; approximately equal to TF-IDF	Optional; not presumed superior.
Ensemble	Small advantage not clearly replicated across seeds	Keep only if paired results justify it.
Recall-first thresholding	Finite-sample rule evaluated; at about 1% FPR, roughly one third of phishing domains still missed	Expose operating point and FPR; do not assume guarantee under arbitrary shift.
Padding robustness	Padding reduced recall; augmentation recovered much of it with some clean-performance cost	Describe as simulated, non-adaptive attack.
Typosquat features	Did not improve benchmark	Record as negative result.
Explanations	Linear explanation ~2 ms/URL; CNN saliency under 1 ms	Report measured latency; SHAP/LIME and human evaluation remain future work.
DNS/TLS/page signals	Absent from data; only assumed-latency cascade analysis	Not implemented; future extension.
Temporal evaluation	No trustworthy timestamps	No temporal/zero-day claim.

10. Deployment, versioning and failure handling

- Package model, feature transformer, normalization rules, threshold policy, class mapping and dependencies together.
- Validate artifact compatibility at startup; if model/vectorizer is missing or mismatched, return a controlled error.
- Use immutable versions. Evaluate a candidate before promotion and retain the prior known-good version for rollback.
- Handle empty, malformed, very long and unsupported inputs with clear error responses.
- Do not automatically visit submitted URLs in the current URL-only prediction path.
- Any future DNS/TLS/page enrichment must be isolated, time-bounded, legally and ethically reviewed, and measured for p50/p95 latency and incremental recall at a fixed FPR.

11. Risks and design decisions

Risk / decision	Impact	Mitigation
URL-only ceiling	High recall may require costly false-positive rates; page/network evidence is absent.	Report recall/FPR trade-off; consider enrichment only after safe collection.
Dataset shortcuts	Source-specific patterns can distort results.	Audit sources/features; use domain-disjoint splits and a second dataset.
Simulated attack realism	Padding tests do not prove robustness to adaptive attackers.	Label transformations simulated/non-adaptive; test adaptive attackers before stronger claims.
Compute limitations	CPU NumPy CNN results do not predict larger neural models.	Treat PyTorch/GPU sequence models as a separate fair comparison.
Threshold under shift	Calibration guarantees may not persist under distribution shift.	Monitor recall and FPR; study recalibration only with suitable target/temporal data.
Multimodal scope	DNS/TLS/page features add latency and privacy/legal complexity.	Decide scope early; claim no multimodal gains until implemented and measured.

12. Implementation roadmap

Phase	Work	Success criterion / dependency
1. Harden evidence	More split seeds; 1,000-resample cluster bootstrap; second public dataset; paired CIs and multiple-comparison handling.	Claims replicate across seeds/datasets or are dropped.
2. Strong sequence baselines	PyTorch CNN/BiLSTM and URL-pretrained Transformer with recall-weighted/focal loss; same domain-disjoint protocol.	Determine whether sequence models beat linear baseline at matched FPR and domain level.
3. Real temporal data	Timestamped phishing first-seen times, dated legitimate snapshots, rolling-origin evaluation and drift monitoring.	Temporal/zero-day estimates only when provenance supports them.
4. Multimodal cascade	DNS age/records, TLS certificates and light page features for deferred URLs; measure p50/p95 latency.	Recall gain per millisecond at fixed FPR; safe crawling and privacy/legal review required.
5. Recall under shift	Cluster-aware calibration, target-feedback recalibration and precision/recall monitoring.	Evaluate coverage and FPR on genuine temporal splits.
6. Explanations	SHAP/LIME latency and faithfulness comparison plus analyst study.	Evidence on speed and usefulness.
7. Paper/release	Report evaluation protocol, recall control, padding-robust training and negative results; release reproducible artifact.	Paper and reproducible package with scoped claims.

13. Conclusion

This design maps the module requirements into an architecture with distinct offline training/evaluation and online prediction paths. The current implementation is URL-only and emphasizes reproducibility, registered-domain-disjoint evaluation, recall-aware thresholding, simulated padding robustness and lightweight explanations. The architecture is extensible, but sequence models and domain/page signals remain future work until implemented and evaluated. The defensible contribution is the measured evidence and evaluation protocol—including negative results—not unsupported claims of novelty or production readiness.

Appendix A. Flowchart interpretation

The central vertical path in Figure 1 is online prediction: URL input, validation, feature extraction, vectorization, model inference, threshold policy, verdict, optional explanation and response. The left-hand path represents offline training: data audit, domain split, training/calibration and model versioning. The right-hand path represents evaluation: domain-disjoint, shift and evasion tests produce metric and latency reports. Keeping these paths separate helps prevent leakage and makes model promotion auditable.